Test Plan for Triangle Counter Programs
Program 3 - PointStore Interface with Text and Binary Encoding
Author: Polina Iaremchuk
Date: February 18th, 2026

All tests run on 2024 MacBook Pro with 14 cores (10 performance and 4 efficiency)

To begin testing the program, you have to be in the src directory.
Then, compile the program with the command "javac com/tryright/*.java".

Overview:
This is the test plan for my right triangle counter programs. The programs
count how many right triangles can be made from a list of points with
integer x,y coordinates.

The algorithm I used is O(n^2) which uses slope grouping instead of checking every
possible combination of 3 points (which would be O(n^3)).
1. Pick each point as a possible right angle vertex
2. Group all other points by their slope from this vertex using a hashmap
3. For each slope, count points with perpendicular slope
4. Multiply counts of perpendicular slope groups
Time: O(n^2)
Space: O(n)

Two slopes are perpendicular when one is the negative reciprocal of the other.
The algorithm normalizes slopes using GCD and encodes them as longs for fast hashmap lookup.

Program 3 adds the PointStore interface for data access abstraction. There are two
implementations:
- TextPointStore: reads text files with point count followed by coordinate pairs
- BinPointStore: reads binary files using memory-mapped I/O (MMIO) where each point is
  encoded as two 4-byte big-endian integers [x, y]

Files ending in .dat use BinPointStore, all others use TextPointStore.

Parameter Tests

Test with 0 parameters (Triangles):
Command: java com.tryright.Triangles
Result: Usage: java com.tryright.Triangles <input_file>
Exit code: 2
Outputs to stderr: yes
Pass: yes

Test with 0 parameters (ProcessTriangles):
Command: java com.tryright.ProcessTriangles
Result: Usage: java com.tryright.ProcessTriangles <input_file> <num_processes>
    input_file: Path to file containing points
    num_processes: Number of processes to use
Exit code: 2
Outputs to stderr: yes
Pass: yes

Test with 0 parameters (ThreadTriangles):
Command: java com.tryright.ThreadTriangles
Result: Usage: java com.tryright.ThreadTriangles <input_file> <num_threads>
Exit code: 2
Outputs to stderr: yes
Pass: yes

Test with 2 parameters (incorrect - second is not a number):
Command: java com.tryright.ProcessTriangles file1 file2
Result: Error: Invalid number of processes - For input string: "file2"
Exit code: 7
Outputs to stderr: yes
Pass: yes

Test with 2 parameters (incorrect - second is not a number):
Command: java com.tryright.ThreadTriangles file1 file2
Result: Error: Invalid number of threads - For input string: "file2"
Exit code: 7
Outputs to stderr: yes
Pass: yes

Functional Tests - Text Encoding

Test 1: testOne.txt
This is the example from the assignment to make sure my algorithm works.
Input:
5
3 4
0 0
3 6
7 4
3 11
Expected: 4
Command: java com.tryright.Triangles test/testOne.txt
Result: 4

Pass: yes

Command: java com.tryright.ProcessTriangles test/testOne.txt 8
Result: 4
Pass: yes

Command: java com.tryright.ThreadTriangles test/testOne.txt 8
Result: 4
Pass: yes

Test 2: testTwo.txt
This tests what happens when there are no right triangles at all. All the points are on the same line so they can't make any triangles.
Input:
8
142 89
284 178
426 267
568 356
710 445
852 534
994 623
1136 712
Expected: 0
Command: java com.tryright.Triangles test/testTwo.txt
Result: 0
Pass: yes

Test 3: testThree.txt
Just 3 points that make one right triangle. This is the minimum case.
Input:
3
-457 823
-457 1205
189 823
Expected: 1
Command: java com.tryright.Triangles test/testThree.txt
Result: 1
Pass: yes

Test 4: testFour.txt
15 random points with multiple right triangles mixed in.
Expected: 5
Command: java com.tryright.Triangles test/testFour.txt

Result: 5
Pass: yes

Test 5: testFive.txt
This tests error handling when the file says there should be 10 points
but only gives 2.
Input:
10
-842 391
573 -628
Expected: Error message
Command: java com.tryright.Triangles test/testFive.txt
Result: Error: Expected 10 points but only found 2
Exit code: 5
The program catches this error and exits properly instead of crashing.
Pass: yes

Test 6: testSix.txt
25 random points spread out to test performance and make sure the algorithm
works with larger datasets.
Expected: 6
Command: java com.tryright.Triangles test/testSix.txt
Result: 6
Pass: yes

Test 7: testProgram3.txt
New test for Program 3 with 7 unique points forming multiple right triangles.
Input:
7
10 20
10 30
20 20
5 15
5 25
15 25
15 15
Expected: 21
Command: java com.tryright.Triangles test/testProgram3.txt
Result: 21
Pass: yes

Command: java com.tryright.ProcessTriangles test/testProgram3.txt 4
Result: 21
Pass: yes

Command: java com.tryright.ThreadTriangles test/testProgram3.txt 4
Result: 21
Pass: yes

Test 8: scaletest.txt
Large dataset with 8245 points for performance testing.
Expected: 150047
Command: java com.tryright.Triangles test/scaletest
Result: 150047
Pass: yes

Command: java com.tryright.ThreadTriangles test/scaletest 8
Result: 150047
Pass: yes

Command: java com.tryright.ProcessTriangles test/scaletest 8
Result: 150047
Pass: yes

Functional Tests - Binary Encoding (.dat files)

Test 9: testBinOne.dat
Binary version of testOne.txt. Same 5 points encoded as big endian integers.
Each point is 8 bytes (4 for x, 4 for y).
Expected: 4
Command: java com.tryright.Triangles test/testBinOne.dat
Result: 4
Pass: yes

Command: java com.tryright.ProcessTriangles test/testBinOne.dat 2
Result: 4
Pass: yes

Command: java com.tryright.ThreadTriangles test/testBinOne.dat 2
Result: 4
Pass: yes

Test 10: testBinTwo.dat
Binary file with 3 points forming a right triangle at origin.
Points: (0,0), (3,0), (0,4)
Expected: 1
Command: java com.tryright.Triangles test/testBinTwo.dat
Result: 1

Pass: yes

Test 11: testBinEmpty.dat
Empty binary file (0 bytes = 0 points).
Expected: 0
Command: java com.tryright.Triangles test/testBinEmpty.dat
Result: 0
Pass: yes

Test 12: testBinNegative.dat
Binary file with negative coordinates.
Points: (-5,-3), (-5,2), (0,-3)
Expected: 1
Command: java com.tryright.Triangles test/testBinNegative.dat
Result: 1
Pass: yes

Binary Format Validation Tests

All tests from Triangles work identically with ThreadTriangles and ProcessTriangles.
The thread-based implementation uses shared memory with int[] array for results.
The process-based implementation uses pipes for inter process communication.
Both use round-robin distribution for load balancing.

Error Cases:
The programs handle these errors:
No filename given: exits with code 2
File doesn't exist: "Error: File not found - [filename]"
File is empty (text): "Error: Empty file"
First line isn't a number (text): "Error: Invalid number format: [content]"
Negative number of points: "Error: Number of points cannot be negative"
Not enough points in file: "Expected X points but only found Y"
Bad coordinate format: "Error: Invalid point format at line X: [content]"
Coordinates aren't integers: "Error: Invalid coordinates at line X: [content]"
Binary file size not multiple of 8: "Invalid binary file format: file size must be a multiple of 8 bytes"

Performance Analysis on MacBook Pro

scaletest (8245 points) with ThreadTriangles:
1 thread: 3.99 seconds
2 threads: 2.15 seconds

4 threads: 1.28 seconds
8 threads: 0.89 seconds
12 threads: 0.71 seconds

Performance improves with more threads up to around 8-12 threads where it plateaus.

Memory-Mapped I/O Performance:
Binary files using MMIO (BinPointStore) show similar performance to text files
for small files but scale better for very large datasets since the OS handles
paging and caching efficiently.

End Of Test Plan